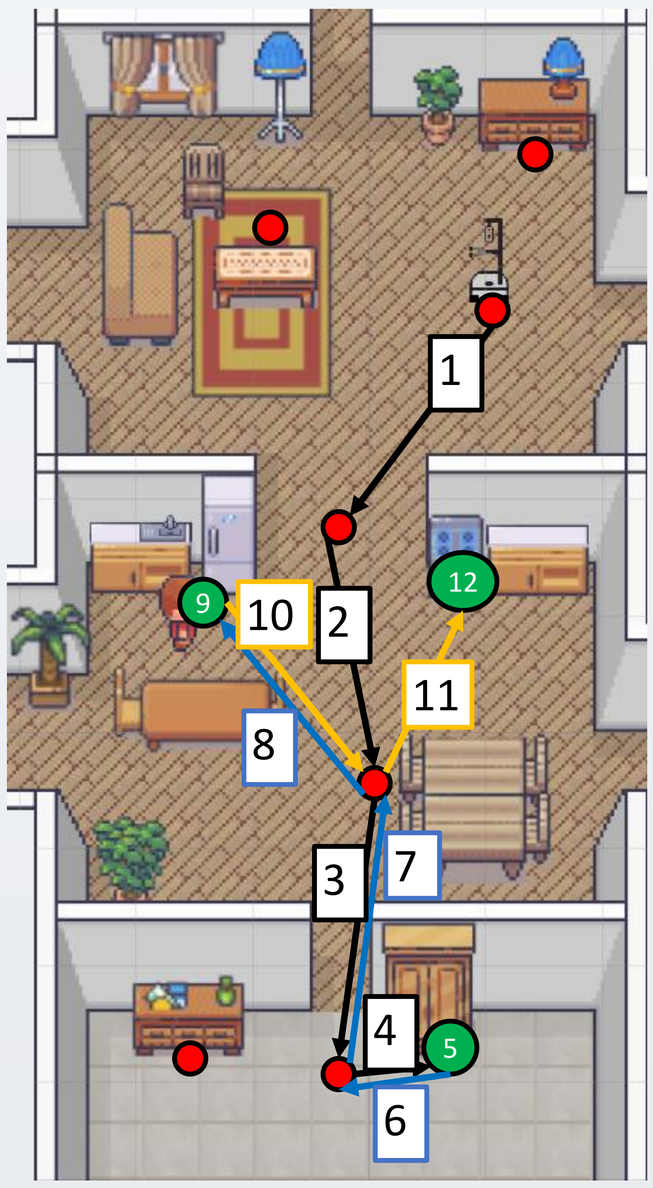
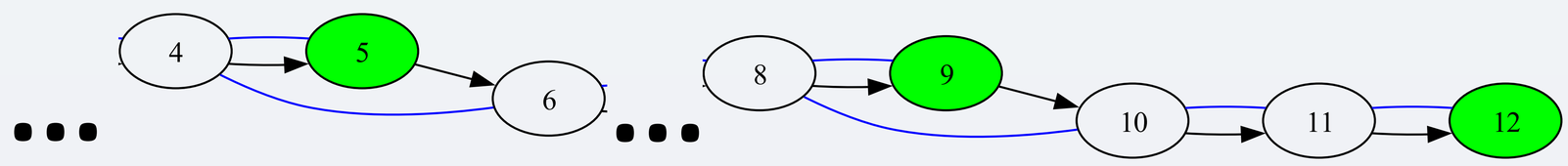
Classical Planning: Dependency Analysis and Plan Obfuscation

Ayman El-Sayed
George Mason University

I created a program that, given problem and domain pddl files, generates a plan list consisting of actions, analyzes dependencies, and flags any actions if they contribute to the goal.

Analysis



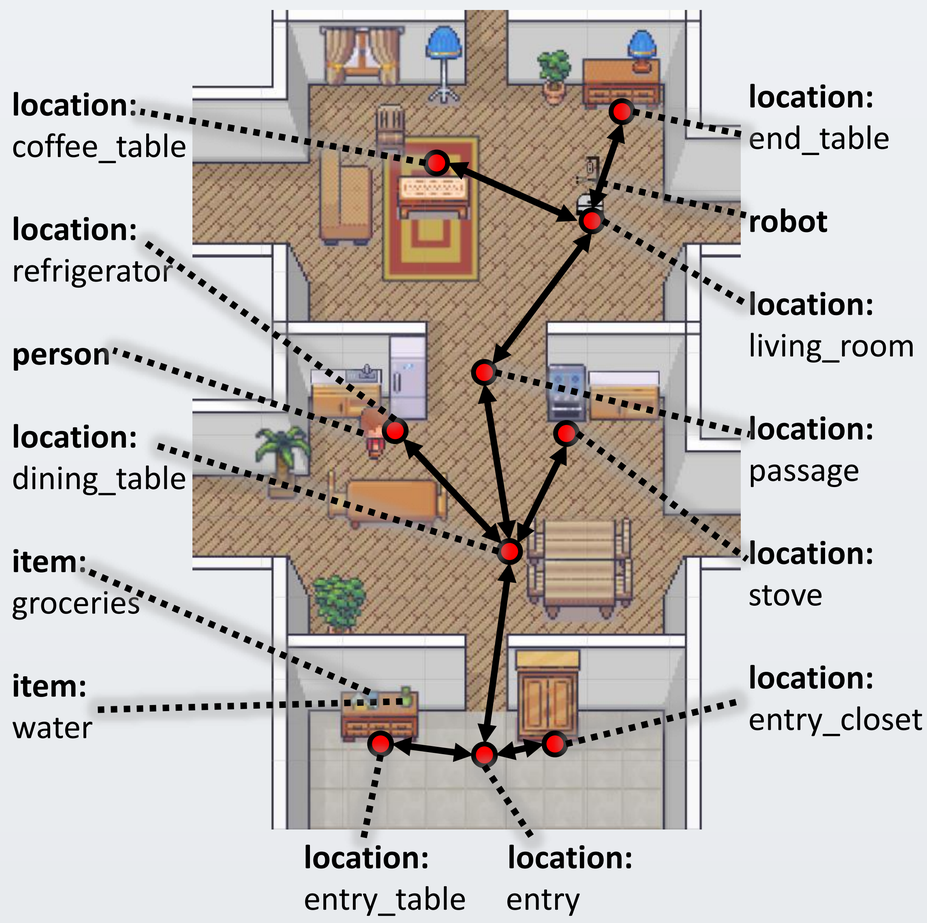


Goal: refrigerator, entry_closet, and stove are open.

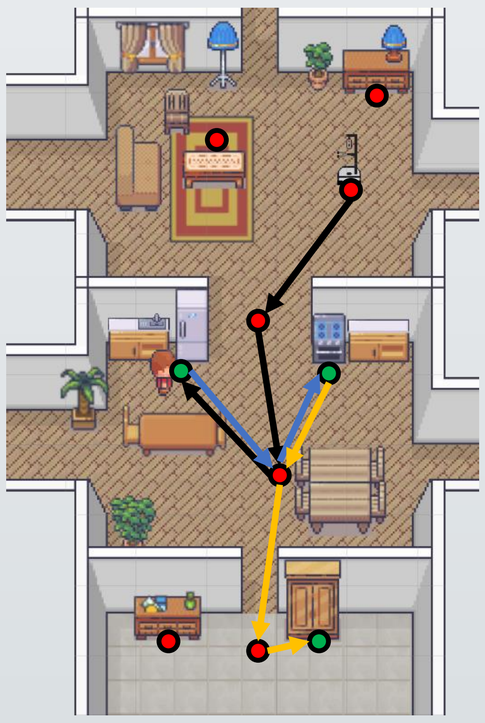
4. move_from_to(entry, entry_closet)
6. move_from_to(entry_closet, entry)
↳ supports each other
5. open_storage(entry_closet)
↳ adds a goal subset

8. move_from_to(dining_table, refrigerator)
10. move_from_to(refrigerator, dining_table)
↳ supports each other
9. open_storage(refrigerator)
12. open_storage(stove)
↳ both add a goal subset

The given problem and domain represents the world shown to the right. The program generated the plan representing the left, with green nodes representing an action that contributed to the subset of the given goal.



Obfuscation



Alternating: If goal can be decomposed into subsets, reorder them and generate each subplan list, then recombine.
Diverting: Pick an action at random. Choose a new action after the picked action that doesn't violate preconditions, then generate new subplan list.

Both methods can be combined if desired.

Example: Goals originally visited in order of entry_closet, refrigerator, stove. Now visited in new order of refrigerator, stove, entry_closet

Closing Remarks

Ongoing Work

- **Multitasking** not yet implemented. This is challenging since it requires actions that contributes to several goals at once.
- Frontend not yet implemented. This is backend work only focusing on plan analysis and manipulation.

Tools Used

- Python with *Unified Planning* and *Fast Downward* planning engine libraries used. Codebase will be released soon.